

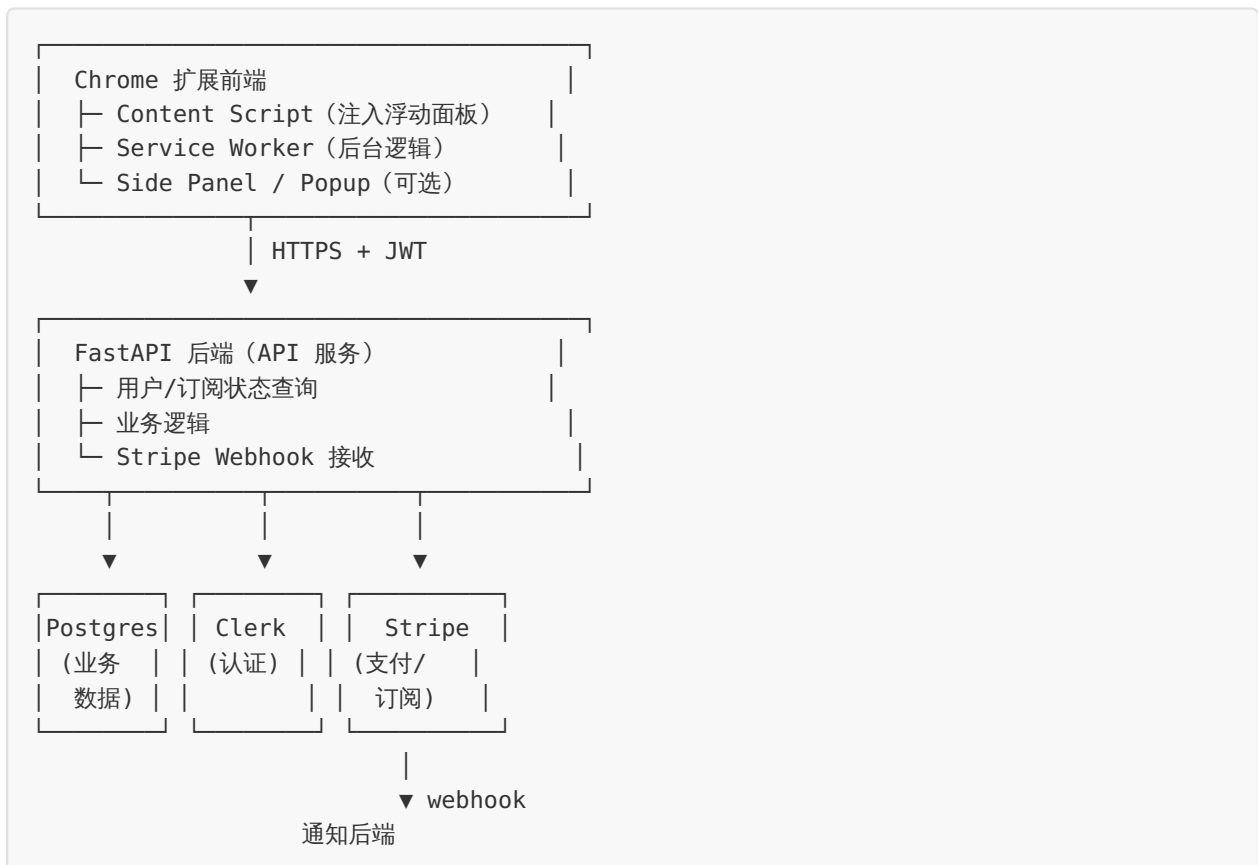
Chrome 插件技术选型文档

项目定位：面向北美用户的 Chrome 浏览器扩展，含用户账号系统与订阅付费功能 **团队规模：**单人开发 **目标：**以最低运维成本快速上线 MVP，支持后续规模化迁移 **更新日期：**2026-05

目录

- 一、整体架构
- 二、技术栈一览
- 三、前端：Chrome 扩展
- 四、后端：FastAPI
- 五、认证：Clerk 或 Supabase Auth
- 六、支付：Stripe
- 七、邮件：Resend
- 八、托管与基础设施
- 九、监控：Sentry
- 十、成本预估
- 十一、合规与安全
- 十二、开发路线图

一、整体架构



核心设计原则:

1. **认证外包给 Clerk:** 自己不存密码、不处理 OAuth、不实现 2FA
2. **支付外包给 Stripe:** 自己不碰信用卡数据、不实现订阅生命周期
3. **后端只管业务:** 用户的订阅状态、业务数据、扩展所需 API
4. **官网 + 扩展双前端:** 注册、订阅管理在网页完成，扩展专注核心功能

二、技术栈一览

层级	选型	理由
扩展前端	原生 JS / TypeScript (Manifest V3)	标准方案，避免框架增加包体积
后端框架	FastAPI (Python 3.11+)	异步原生、自动文档、类型友好
ORM	SQLAlchemy 2.0 + Alembic (迁移)	生态最成熟
数据库	PostgreSQL 15+	关系型 + JSON 字段灵活，事实标准
认证	Clerk (首选) / Supabase Auth (预算敏感)	省去密码管理、社交登录、2FA 实现
支付	Stripe (Checkout + Customer Portal + Webhook)	北美事实标准，开发体验最佳
邮件	Resend	现代 API、免费额度够用、送达率好
托管	Render / Railway / Fly.io	PaaS 三选一，按熟悉度选
监控	Sentry	错误追踪行业标准
CDN/DNS	Cloudflare	免费、DNS 管理方便
域名	Cloudflare Registrar / Namecheap	不加价

三、前端：Chrome 扩展

3.1 Manifest V3 基础配置

```
{
  "manifest_version": 3,
  "name": "Your Extension Name",
  "version": "0.1.0",
  "description": "Your description",
  "action": {
    "default_title": "Open"
  },
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "js": ["content.js"],
    "css": ["content.css"],
    "run_at": "document_idle"
  }],
  "permissions": ["storage", "tabs", "scripting"],
  "host_permissions": [
    "https://api.yourapp.com/*"
  ],
  "externally_connectable": {
    "matches": ["https://yourapp.com/*"]
  },
  "side_panel": {
    "default_path": "sidepanel.html"
  }
}
```

3.2 UI 形态选择

- **浮动面板**（content script 注入）——主交互界面
- **Side Panel API**（Chrome 114+）——固定侧边栏，跨标签页保留
- **Popup**——快速操作（设置、登录入口）
- **不要做**：跳出浏览器置顶的悬浮窗（扩展无此权限）

3.3 登录流程（与 Clerk 配合）

用户在扩展点登录 → 打开网站登录页 → Clerk 处理登录 → 网站把 token 回传扩展 → 扩展存储

```
// 在扩展面板里发起登录
chrome.tabs.create({
  url: 'https://yourapp.com/sign-in?source=extension&ext_id=' + chrome.runtime.id
});

// 在你的网站登录成功后, 向扩展发送 token
// (在 yourapp.com 的页面 JS 里执行)
chrome.runtime.sendMessage(EXTENSION_ID, {
  type: 'auth',
  token: clerkSessionToken
});

// 在扩展 background.js 里接收
chrome.runtime.onMessageExternal.addListener(
  (msg, sender, sendResponse) => {
    if (msg.type === 'auth') {
      chrome.storage.local.set({ token: msg.token });
      sendResponse({ ok: true });
    }
  }
);
```

3.4 调用后端 API

```
async function apiCall(path, options = {}) {
  const { token } = await chrome.storage.local.get('token');
  if (!token) throw new Error('未登录');

  return fetch(`https://api.yourapp.com${path}`, {
    ...options,
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
      ...(options.headers || {})
    }
  });
}
```

四、后端：FastAPI

4.1 项目结构

```

backend/
├── app/
│   ├── main.py           # 入口
│   ├── config.py        # pydantic-settings 配置
│   ├── database.py       # SQLAlchemy 引擎/Session
│   ├── deps.py          # 依赖注入 (认证、DB session)
│   ├── models/          # SQLAlchemy 模型
│   │   ├── user.py
│   │   └── subscription.py
│   ├── schemas/         # Pydantic schema
│   ├── routers/
│   │   ├── auth.py
│   │   ├── billing.py    # Stripe Checkout/Portal
│   │   ├── webhooks.py   # Stripe webhook 接收
│   │   └── api.py        # 业务 API
│   └── services/
│       ├── stripe_service.py
│       └── clerk_service.py
├── alembic/             # 数据库迁移
├── pyproject.toml
└── .env
  
```

4.2 关键依赖

```

[project]
dependencies = [
    "fastapi>=0.110",
    "uvicorn[standard]>=0.27",
    "sqlalchemy>=2.0",
    "alembic>=1.13",
    "psycopg[binary]>=3.1",
    "pydantic-settings>=2.0",
    "python-jose[cryptography]>=3.3", # JWT 验证
    "stripe>=7.0",
    "resend>=0.7",
    "sentry-sdk[fastapi]>=1.40",
    "httpx>=0.26",
]
  
```

4.3 配置管理

```
# app/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    # 数据库
    database_url: str

    # Clerk
    clerk_secret_key: str
    clerk_publishable_key: str
    clerk_jwt_public_key: str # 用于验证 JWT

    # Stripe
    stripe_secret_key: str
    stripe_webhook_secret: str
    stripe_price_id_monthly: str
    stripe_price_id_yearly: str

    # 邮件
    resend_api_key: str

    # 域名
    frontend_url: str = "https://yourapp.com"
    cors_origins: list[str] = []

    # 监控
    sentry_dsn: str | None = None

settings = Settings()
```

4.4 CORS 与扩展通信

```
# app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import settings

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        settings.frontend_url,
        "chrome-extension://YOUR_EXTENSION_ID", # 扩展 ID
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

4.5 认证依赖（验证 Clerk JWT）

```
# app/deps.py
from fastapi import Depends, HTTPException, Header
from jose import jwt, JWTError
from app.config import settings

async def get_current_user(authorization: str = Header(...)):
    if not authorization.startswith("Bearer "):
        raise HTTPException(401, "Invalid auth header")

    token = authorization[7:]
    try:
        payload = jwt.decode(
            token,
            settings.clerk_jwt_public_key,
            algorithms=["RS256"],
            options={"verify_aud": False}
        )
        return payload # 含 user_id 等信息
    except JWTError:
        raise HTTPException(401, "Invalid token")
```

五、认证：Clerk 或 Supabase Auth

5.1 选型对比

维度	Clerk	Supabase Auth
免费额度	月活 1 万	月活 5 万
开发体验	业内最佳，UI 组件漂亮	不错，跟数据库一起用更顺
社交登录	Google/Apple/GitHub 等开箱即用	同样支持
2FA / MFA	内置	内置
企业 SSO	高价档支持	Pro 档有限支持
自定义程度	较高	高
适合场景	重视用户体验、不缺预算	数据库已用 Supabase、预算敏感

推荐：MVP 阶段用 **Clerk**，1 万 MAU 内免费，省下的开发时间最值。

5.2 数据流转

- 1. 用户在网站点登录 → Clerk 托管 UI 完成认证
- 2. Clerk 颁发 JWT（短期，约 1 小时）+ 刷新机制

3. 网站把 JWT 回传扩展，扩展存到 `chrome.storage.local`
4. 扩展调用 API 时把 JWT 放进 Authorization header
5. FastAPI 用 Clerk 公钥验证 JWT，解析出 `user_id`
6. 业务表里用 Clerk 的 `user_id` 作外键

5.3 用户数据同步

Clerk 管认证，但**业务数据要存自己的数据库**。两种方式同步：

方案 A：webhook（推荐） Clerk 在用户注册/更新时调你的 webhook，你写入本地 `users` 表。

方案 B：懒同步 每次 API 请求时检查本地是否有该 `user_id`，没有就查 Clerk API 拉取后写入。

5.4 用户表结构示例

```
# app/models/user.py
from sqlalchemy.orm import Mapped, mapped_column
from datetime import datetime

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    clerk_user_id: Mapped[str] = mapped_column(unique=True, index=True)
    email: Mapped[str] = mapped_column(index=True)
    stripe_customer_id: Mapped[str | None] = mapped_column(unique=True)
    subscription_status: Mapped[str] = mapped_column(default="free")
    subscription_tier: Mapped[str | None] = mapped_column()
    subscription_period_end: Mapped[datetime | None] = mapped_column()
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

六、支付：Stripe

6.1 集成方式：Checkout + Customer Portal + Webhook

不要自己写支付表单。用 Stripe 托管的 Checkout 页面（PCI 合规他们处理），用户管理订阅用 Customer Portal（Stripe 提供完整 UI）。

6.2 关键流程

创建 Checkout Session（用户点订阅）

```
# app/routers/billing.py
import stripe
from fastapi import APIRouter, Depends
from app.config import settings
from app.deps import get_current_user

stripe.api_key = settings.stripe_secret_key
router = APIRouter(prefix="/billing")

@router.post("/checkout")
async def create_checkout(
    plan: str, # "monthly" / "yearly"
    user = Depends(get_current_user)
):
    price_id = (
        settings.stripe_price_id_monthly if plan == "monthly"
        else settings.stripe_price_id_yearly
    )

    session = stripe.checkout.Session.create(
        mode="subscription",
        line_items=[{"price": price_id, "quantity": 1}],
        success_url=f"{settings.frontend_url}/success?session_id={{CHECKOUT_SESSION_ID}}",
        cancel_url=f"{settings.frontend_url}/pricing",
        client_reference_id=user["sub"], # Clerk user_id
        customer_email=user.get("email"),
        allow_promotion_codes=True,
    )
    return {"url": session.url}
```

Customer Portal（用户管理订阅）

```
@router.post("/portal")
async def create_portal(user = Depends(get_current_user)):
    # 假设已经在 user 表存了 stripe_customer_id
    db_user = get_user_by_clerk_id(user["sub"])

    portal = stripe.billing_portal.Session.create(
        customer=db_user.stripe_customer_id,
        return_url=f"{settings.frontend_url}/account",
    )
    return {"url": portal.url}
```

Webhook 接收（订阅状态以此为准）

```
# app/routers/webhooks.py
from fastapi import APIRouter, Request, HTTPException
import stripe

router = APIRouter()

@router.post("/webhooks/stripe")
async def stripe_webhook(request: Request):
    payload = await request.body()
    sig = request.headers.get("stripe-signature")

    try:
        event = stripe.Webhook.construct_event(
            payload, sig, settings.stripe_webhook_secret
        )
    except (ValueError, stripe.error.SignatureVerificationError):
        raise HTTPException(400, "Invalid signature")

    # 关键事件
    if event["type"] == "checkout.session.completed":
        session = event["data"]["object"]
        clerk_user_id = session["client_reference_id"]
        customer_id = session["customer"]
        # 更新 user 表的 stripe_customer_id

    elif event["type"] in (
        "customer.subscription.created",
        "customer.subscription.updated",
    ):
        sub = event["data"]["object"]
        # 更新订阅状态、当前周期结束时间

    elif event["type"] == "customer.subscription.deleted":
        sub = event["data"]["object"]
        # 标记为取消

    return {"received": True}
```

6.3 必须打开的 Stripe 功能

- **Stripe Tax**: 自动计算和收取北美销售税（约 0.5% 费率，但省掉合规复杂度）
- **Customer Portal**: 在 Dashboard 启用并配置允许的操作（取消、改卡、查发票）
- **Webhook 端点**: 在 Dashboard 配置 → 复制 webhook secret 到 `.env`

6.4 计费费率

- 北美刷卡: **2.9% + \$0.30** 每笔
- 国际卡: 再加 **1%~2%**
- Stripe Tax: **+0.5%**

- 退款：扣除手续费不退还

七、邮件：Resend

7.1 用途

- 注册欢迎邮件
- 订阅成功 / 取消通知
- 支付失败提醒
- 密码重置（如果不用 Clerk）

7.2 集成

```
# app/services/email.py
import resend
from app.config import settings

resend.api_key = settings.resend_api_key

async def send_welcome(to: str, name: str):
    resend.Emails.send({
        "from": "Your App <hello@yourapp.com>",
        "to": [to],
        "subject": "Welcome!",
        "html": f"<h1>Hi {name}</h1><p>Welcome aboard.</p>",
    })
```

7.3 域名验证

- 在 Resend Dashboard 添加你的发件域名
- 按提示在 DNS（Cloudflare）添加 SPF/DKIM 记录
- 验证通过后才能用该域名发件

重要：不要用 `gmail.com` 之类做发件人，会进垃圾箱。准备好自己的域名 + 子域（如 `mail.yourapp.com`）。

7.4 免费额度

- 每月 3000 封 + 每天 100 封
 - 早期完全够用，超出后 \$20/月起 5 万封
-

八、托管与基础设施

8.1 三选一推荐

平台	优点	起步月费
Render	UI 最简单、文档好、部署快	~\$14 (Web + DB)
Railway	配置极简、按用量计费	~\$10~\$15
Fly.io	全球节点多、冷启动快	~\$5~\$10

MVP 推荐 Render：界面友好，新手最快上手。

8.2 部署清单

- [] Web Service (FastAPI + uvicorn/gunicorn)
- [] PostgreSQL 数据库
- [] 环境变量配置 (所有 secrets)
- [] 自定义域名 (`api.yourapp.com`)
- [] HTTPS (PaaS 自动配 Let's Encrypt)
- [] 健康检查端点 `/health`
- [] Stripe webhook URL 配置 (指向部署后的地址)

8.3 域名规划

- `yourapp.com` → 营销官网 + 用户登录注册
- `api.yourapp.com` → FastAPI 后端
- `mail.yourapp.com` → Resend 发件子域

域名注册推荐 **Cloudflare Registrar** (不加价) + Cloudflare DNS (免费 + 快)。

8.4 启动命令示例

```
# 生产环境 (Render Build Command)
pip install -r requirements.txt
alembic upgrade head

# Start Command
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

九、监控：Sentry

9.1 集成

```
# app/main.py
import sentry_sdk
from sentry_sdk.integrations.fastapi import FastApiIntegration

if settings.sentry_dsn:
    sentry_sdk.init(
        dsn=settings.sentry_dsn,
        integrations=[FastApiIntegration()],
        traces_sample_rate=0.1, # 性能采样 10%
        environment="production",
    )
```

9.2 关键告警

- 5xx 错误率突增
- Stripe webhook 处理失败
- 数据库连接异常
- 慢查询 (>1s)

9.3 免费额度

每月 5000 个错误事件，1 个用户席位。早期够用。

十、成本预估

10.1 各阶段月成本

阶段	用户量	月成本
开发期	0	~\$0~\$14
上线初期	<100 付费用户	~\$15~\$30
增长期	1000 付费 / 月流水 \$5K	~\$50~\$100 + Stripe 抽成
规模化	1 万+ 付费	\$300~\$1000 + 抽成

10.2 一次性 / 年费成本

- 域名：~\$12/年

- Apple/Google 开发者账号（如未来扩展平台）：暂不需要
- Chrome Web Store 一次性发布费：\$5

10.3 何时升级

服务	触发升级的信号
Clerk	月活 > 1 万
Supabase	数据库 > 500MB 或项目暂停
Resend	月发件 > 3000
Sentry	月事件 > 5000
Render	内存/CPU 持续打满

十一、合规与安全

11.1 必备文档

- **隐私政策**（Privacy Policy）—— Chrome Web Store 强制要求
- **服务条款**（Terms of Service）
- **退款政策**（Refund Policy）—— 订阅服务必备

可用模板：[Termly](#)、[GetTerms](#) 生成。

11.2 用户数据声明（Chrome Web Store 提交时填写）

- 收集了什么数据（邮箱、使用统计等）
- 用途说明
- 是否分享给第三方（Stripe、Clerk 等都要列出）
- 是否加密传输

11.3 GDPR / CCPA

- 用户数据导出接口（API 提供 `/me/export`）
- 账号删除接口（连带删除 Clerk 用户和 Stripe 客户）
- Cookie 同意（如官网用追踪）

Clerk 和 Stripe 都签署了 DPA（数据处理协议），帮你扛大半合规责任。

11.4 安全要点

- 所有 secrets 用环境变量管理，不进 Git

- `.env` 加入 `.gitignore`
 - Stripe webhook 必须验证签名
 - JWT 有过期时间 (Clerk 默认 1 小时)
 - 数据库连接用 SSL
 - 定期备份数据库 (Render/Railway 自带, 但额外做异地备份)
 - Manifest V3 不允许远程代码执行——遵守即可
-

十二、开发路线图

Phase 1: MVP 基础 (1~2 周)

- ☐ FastAPI 项目骨架 + Alembic 迁移
- ☐ 本地 PostgreSQL 跑通
- ☐ Clerk 注册账号、配置基础登录
- ☐ 实现 `/me` 接口 (验证 JWT、返回用户信息)
- ☐ 扩展 Manifest V3 骨架
- ☐ content script 浮动面板

Phase 2: 核心功能 (2~4 周)

- ☐ 业务 API 完成
- ☐ 扩展前端调用 API
- ☐ 错误处理 + 加载状态

Phase 3: 付费集成 (1 周)

- ☐ Stripe 账号 + 产品/价格配置
- ☐ Checkout Session 接口
- ☐ Customer Portal 接口
- ☐ Webhook 接收 + 用户表订阅状态字段
- ☐ 扩展里的"升级 Pro"按钮 + 功能门禁

Phase 4: 上线准备 (1 周)

- ☐ 部署到 Render (后端 + 数据库)
- ☐ 域名 + DNS 配置
- ☐ Resend 域名验证 + 邮件模板
- ☐ Sentry 接入
- ☐ 隐私政策/服务条款页面

- ☐ 营销官网（落地页 + 定价页）

Phase 5: 发布

- ☐ Chrome Web Store 开发者账号 (\$5 一次性)
- ☐ 准备截图、宣传图、描述
- ☐ 提交审核（通常 1~3 天）
- ☐ Product Hunt / Hacker News 发布

Phase 6: 迭代

- ☐ 用户反馈渠道（邮件 / Discord / Tally 表单）
- ☐ 数据分析（PostHog 免费档 / Plausible）
- ☐ A/B 测试定价
- ☐ 持续优化转化漏斗

附录：参考资源

- FastAPI 官方文档：<https://fastapi.tiangolo.com>
- Clerk 文档：<https://clerk.com/docs>
- Stripe 文档：<https://stripe.com/docs>
- Chrome Extension 文档：<https://developer.chrome.com/docs/extensions>
- Manifest V3 迁移指南：<https://developer.chrome.com/docs/extensions/develop/migrate>

文档使用建议：把这份文档放到项目仓库的 `docs/tech-stack.md`，每次选型变更或踩坑发现都更新一段。半年后回看，它会变成你最值钱的资产之一。